ROS2

Behavior Tree

Cesar Bastos

Unicamp, Brazil

October 22, 2026

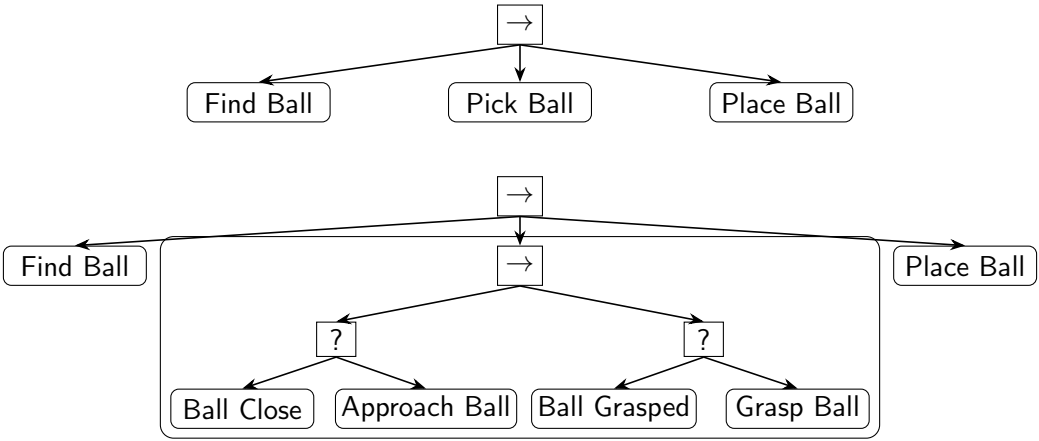
Question

Think about this

How do behavior trees organize and control the execution of behaviors in an autonomous system?

- A robot must find, pick up, and place a ball. After finding it, the robot approaches until the ball is close, then tries to grasp it until it is secure. Once the ball is grasped, the robot moves on to place it. Each step must succeed before the robot starts the next one.

Behavior tree: Example



Expanded Pick Ball sub-tree

What is a behavior tree?

Definition

Autonomous agents use behavior models to organize their possible actions and decide when each action should run. A behavior tree represents this decision process as a hierarchy of conditions and actions, with rules that control the order of execution and how the agent switches between behaviors.

- Mathematical Model of Plan Execution – describe switching between a finite set of tasks in a modular fashion
- Originated from Game Industry, as a powerful way to describe AI for "NPC"
- **Advantage:** BTs have also made it into the robotics domain as robots have become increasingly capable of doing more than simple repetitive tasks

What is a behavior tree?

Behavior trees (BTs) are defined by the following characteristics:

- **Behavior trees:** They start at a root node and are designed to be traversed in a specific order until a terminal state is reached (success or failure).
- **Leaf nodes are executable behaviors:** Each leaf will do something, whether it's a simple check or a complex action, and will output a status (success, failure, or running). In other words, leaf nodes are where you connect a BT to the lower-level code for your specific application.
- **Internal nodes control tree traversal:** The internal (non-leaf) nodes of the tree will accept the resulting status of their children and apply their own rules to dictate which node should be expanded next.

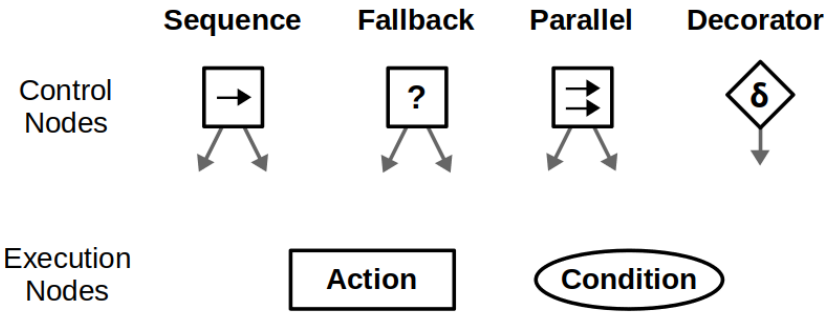


Behavior Tree Terminology

Terminology

Terminology varies across the literature and software libraries. In this presentation, I follow the definitions in **Behavior Trees in Robotics and AI: An Introduction** by Colledanchise and Ögren.

Behavior Tree Terminology



The execution of a BT starts from the root which sends ticks with a certain frequency to its child. A tick is an enabling signal that allows the execution of a child. When the execution of a node in the BT is allowed, it returns to the parent a status **running** if its execution has not finished yet, **success** if it has achieved its goal, or **failure** otherwise.

Behavior Tree Terminology

Execution nodes are the leaves of a behavior tree, and are either Condition nodes or Action nodes.

- A **condition** checks something and returns Success or Failure during the same tick. In ROS 2, for example, a condition could call a service such as **/gripper/get_state** to check whether the gripper is open.
- An **action** performs a task that may take longer than one tick. For example, a BT node could send a navigation goal using ROS 2's *NavigateToPose* action. It returns Running while the robot is moving, then returns Success or Failure when the task finishes. ROS 2 actions provide a suitable interface for tasks that run over time.

Behavior Tree Terminology

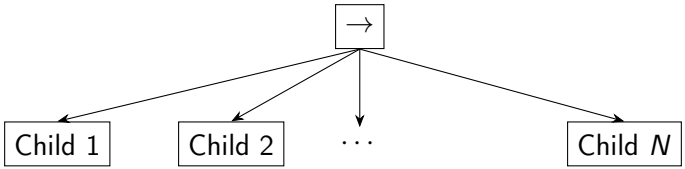
Control nodes are internal nodes and define how to traverse the BT given the status of their children.

- Importantly, children of control nodes can be execution nodes or control nodes themselves. Sequence, Fallback, and Parallel nodes can have any number of children, but differ in how they process said children. Decorator nodes necessarily have one child, and modify its behavior with some custom defined policy.

Behavior Tree Terminology

The **Sequence** node executes the following pseudocode, which corresponds to routing the ticks to its children from the left until it finds a child that returns either Failure or Running, then it returns Failure or Running accordingly to its own parent. It returns Success if and only if all its children return Success. Note that when a child returns Running or Failure, the Sequence node does not route the ticks to the next child (if any). The symbol of the Sequence node is a box containing the label "$\rightarrow$".

Behavior Tree Terminology



A sequence node with N children

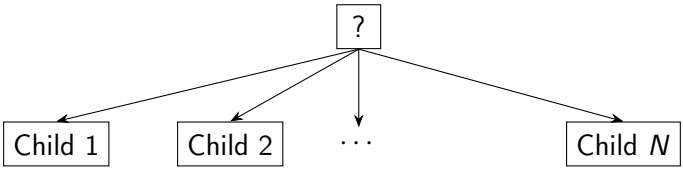
Sequence node pseudocode

```
1  for i ← 1 to N do
2      childStatus ← Tick(child(i))
3      if childStatus = Running then
4          return Running
5      else if childStatus = Failure then
6          return Failure
7  return Success
```

Behavior Tree Terminology

The **Fallback** node executes the following pseudocode, which corresponds to routing the ticks to its children from the left until it finds a child that returns either *Success* or *Running*, then it returns *Success* or *Running* accordingly to its own parent. It returns *Failure* if and only if all its children return *Failure*. Note that when a child returns *Running* or *Success*, the Fallback node does not route the ticks to the next child (if any). The symbol of the the Fallback node is a box containing the label "?".

Behavior Tree Terminology



A fallback node with N children

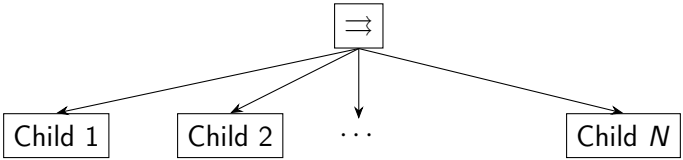
Fallback node pseudocode

```
1  for i ← 1 to N do
2     childStatus ← Tick(child(i))
3     if childStatus = Running then
4        return Running
5     else if childStatus = Success then
6        return Success
7  return Failure
```

Behavior Tree Terminology

The **Parallel** node executes the following pseudocode, which corresponds to routing the ticks to all its children and it returns Success if M children return Success, it returns Failure if N $-$ M $+1$ children return Failure, and it returns Running otherwise, where N is the number of children and M $\leq$ N is a user defined threshold. The symbol of the the Parallel node is a box containing the label "$\rightrightarrows$".

Behavior Tree Terminology

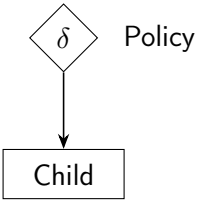


A parallel node with N children

Parallel node pseudocode with success threshold M

```
1  for i ← 1 to N do
2      childStatus(i) ← Tick(child(i))
3  if Σ_{i: childStatus(i)=Success} 1 ≥ M then
4      return Success
5  else if Σ_{i: childStatus(i)=Failure} 1 > N − M then
6      return Failure
7  return Running
```

Behavior Tree Terminlogy



The diamond label names the policy.

A **Decorator** has one child. It can change the child's return status and control when the child is ticked.

Examples

- **Invert:** changes Success to Failure and Failure to Success.
- **Max-N-tries:** after N failures, returns Failure without ticking the child again.
- **Max-T-sec:** if the child is still Running after T seconds, returns Failure and stops ticking it.

Behavior Tree Terminology

Node type	**Symbol**	**Succeeds**	**Fails**	**Running**
Fallback	?	If one child succeeds	If all children fail	If one child returns Running
Sequence	→	If all children succeed	If one child fails	If one child returns Running
Parallel	⇉	If $\geq M$ children succeed	If $> N - M$ children fail	Else
Action	text	Upon completion	If impossible to complete	During completion
Condition	text	If true	If false	Never
Decorator	◇	Custom	Custom	Custom

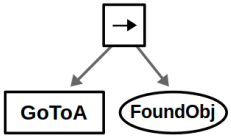
Table 1.1: The node types of a BT.

Example

Suppose we have a mobile robot that must search for specific objects in a home environment. Assume the robot knows all the search locations beforehand; in other words, it already has a world model to operate in.

Example

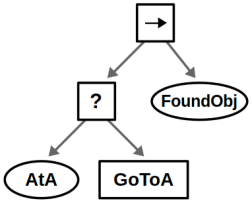
Let's start simple. If there is only one location (we'll call it A), then the BT is a simple sequence of the necessary actions: Go to the location and then look for the object.



The navigation is represented as an action node, as it may take some time for the robot to move (returning Running in the process). On the other hand, the vision is represented as a condition node, assuming the robot can detect the object from a single image once it arrives at its destination

Example

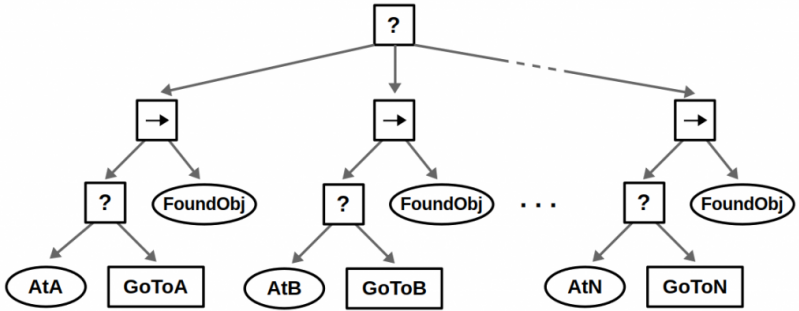
One very common design principle you should know is defined in the book as explicit success conditions. In simpler terms, you should almost always check before you act. For example, if you're already at a specific location, why not check if you're already there before starting a navigation action?



Explicit success conditions use a Fallback node with a condition preceding an action. The guarded action will only execute if the success condition is not met — in this example if the robot is not at location A

Example

Our robot likely operates in an environment with multiple locations, and the idea is to look in all possible locations until we find the object of interest. This can be done by introducing a root-level Fallback node and repeating the above behavior for each location in some specified order.

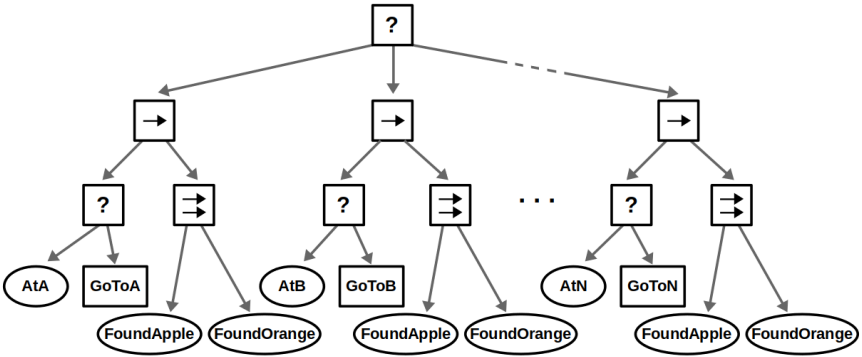


Example

Finally, suppose that instead of looking for a single object, we want to consider several objects, let's say apples and oranges. This use case of composing conditions can be done with Parallel nodes as shown below.

- If we accept either an apple or an orange ("OR" condition), then we succeed if one node returns Success.
- If we require both an apple and an orange ("AND" condition), then we succeed if both nodes return Success.
- If we care about the order of objects, e.g., you must find an apple before finding an orange, then this could be done with a Sequence node instead.

Example


?
?
?
?
?
AtA
GoToA
FoundApple
FoundOrange
AtB
GoToB
FoundApple
FoundOrange
AtN
GoToN
FoundApple
FoundOrange

Behavior trees vs. finite-state machines

- FSMs and BTs have equivalent expressive power, but FSMs can become impractical as behavior grows because they require many transitions.
- FSM transitions can conflict: multiple outgoing conditions from one state may be true at the same time. Making them mutually exclusive requires carefully revising the transitions.
- FSMs are difficult to maintain at scale. Adding a state can require rechecking existing transitions and adding new ones. Dense transition graphs also make automated analysis and synthesis computationally expensive.
- HFSMs are closer to BTs because both organize complex behavior into modules and can represent parallel activities.
- Their modularity differs: HFSMs require each hierarchy level to be defined explicitly. A BT subtree can stand alone as a module, with the same interface as an atomic action.

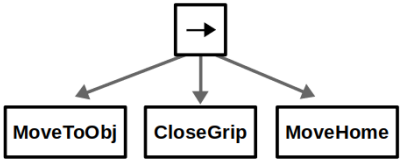
Behavior trees vs. finite-state machines

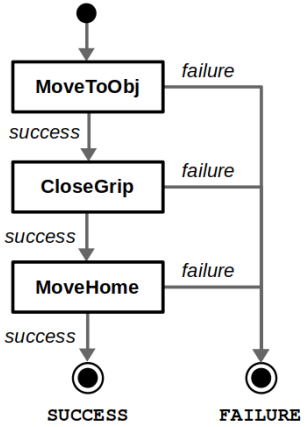
Sumarizing:

- In theory, it is possible to express anything as a BT, FSM, one of the other abstractions, or as plain code. However, each model has its own advantages and disadvantages in their intent to aid design at larger scale.
- Specific to BTs vs. FSMs, there is a tradeoff between modularity and reactivity. Generally, BTs are easier to compose and modify while FSMs have their strength in designing reactive behaviors.

Behavior trees vs. finite-state machines

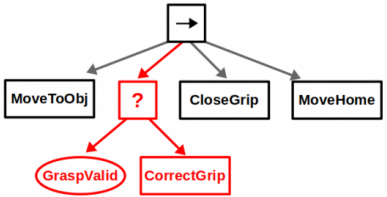
Suppose we have a picking task where a robot must move to an object, grab it by closing its gripper, and then move back to its home position. For a simple design like this, both implementations are relatively clean and easy to follow.

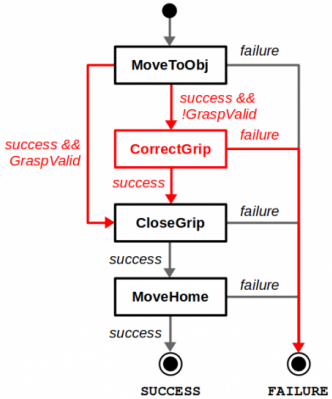




Behavior trees vs. finite-state machines

And if we want to modify this behavior? Say we first want to check whether the pre-grasp position is valid, and correct if necessary before closing the gripper. With a BT, we can directly insert a subtree along our desired sequence of actions, whereas with a FSM we must rewire multiple transitions. This is what we mean when we claim BTs are great for modularity.



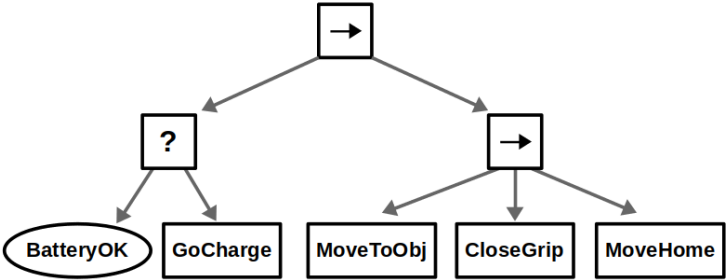


Potential Issues with Behavior Trees

- A fully reactive behavior, such as interrupting any task to recharge when the battery is low, may be easier to represent with an FSM, although the state diagram can become cluttered.
- A behavior tree typically starts evaluation at the root node on each tick.
- As the tree grows deeper, repeatedly traversing it from the root can become less efficient.
- An engine can track active nodes and tick them directly, reducing the need to traverse the entire tree on every tick.

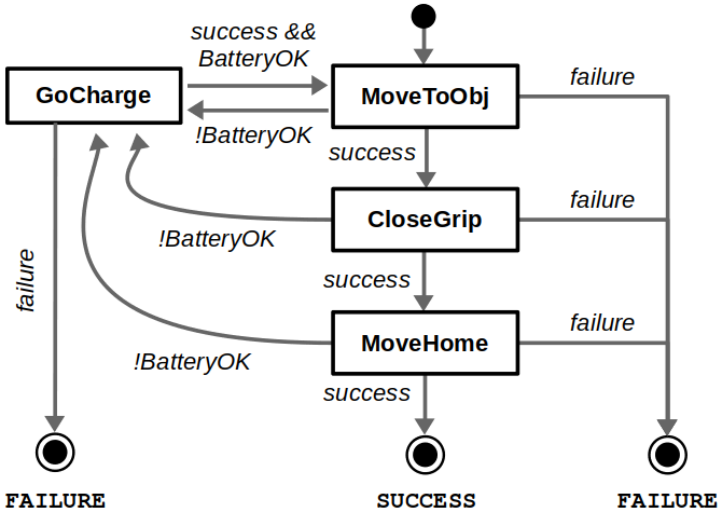
Battery reactivity

Behavior Tree:



Battery reactivity

FSM:



Exploring Nav2's Behavior Tree

- Nav2 uses a behavior tree to coordinate navigation actions and recovery behaviors.
- We can open the tree in Groot2 to inspect how its nodes are organized.

Default XML file for `NavigateToPose`

`nav2_bt_navigator/behavior_trees/navigate_to_pose_w_replanning_and_recovery.xml`

In an installed ROS 2 system, find the package directory with `ros2 pkg prefix nav2_bt_navigator`, then look in its `share/nav2_bt_navigator/behavior_trees` folder.

Useful tutorials

Nav2 tutorials

- Adding Your Task Server to the Lifecycle Node
- Detailed Behavior Tree Walkthrough
- Writing a New Behavior Tree Plugin
- Plugin Example

Behavior Tree Control Nodes

- **PipelineSequence:** Ticks children in order and re-ticks earlier children while a later child is *Running*. Nav2 uses it to support replanning while following a path.
- **ReactiveSequence:** Starts from the first child on each tick, rechecking conditions. If a condition fails, it halts the running action that follows it.
- **RoundRobin:** Tries its children in turn until one succeeds. Nav2 uses it to cycle through recovery actions.
- **ReactiveFallback:** Checks children in priority order, starting from the first on each tick. If a higher-priority child succeeds, it halts the lower-priority running branch.

References


K.-S. Chang and D. Zhu, *Hierarchical Finite State Machine (HFSM) & Behavior Tree (BT)*, CS123, Lecture 8, Stanford University, 2015.


M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*, arXiv:1709.00084, 2017.


S. Castro, *Introduction to Behavior Trees*, RoboHub, 2021.